

图的世界

图论入门 · 8题 · C++ & Python 双语对照

NQ147 拓扑排序试炼之有向图的拓扑序列 AcWing 848

给定一个 n 个点 m 条边的有向图，点的编号是1到 n ，图中可能存在重边和自环。请输出任意一个该有向图的拓扑序列，如果拓扑序列不存在，则输出-1。若一个由图中所有点构成的序列 A 满足：对于图中的每条边 (x, y) ， x 在 A 中都出现在 y 之前，则称 A 是该图的一个拓扑序列。数据范围： $1 \leq n, m \leq 10^5$

输入

第一行包含两个整数 n 和 m 接下来 m 行，每行包含两个整数 x 和 y ，表示存在一条从点 x 到点 y 的有向边 (x, y) 。

输出

共一行，如果存在拓扑序列，则输出拓扑序列。 否则输出-1。

来源

AcWing 848

输入

```
3 3
1 2
2 3
1 3
```

输出

```
1 2 3
```

C++

```

#include <iostream>
#include <algorithm>
#include <queue>
#include <cstring>

using namespace std;
#define For(a, begin, end) for (int a = (begin); a < (end); a++)
const int N = 1000007;

int n, m;
int head[N], edge[N], nextVertex[N], idx; //邻接表
int q[N], d[N]; //队列, 入度

void add(int a, int b)
{
    edge[idx] = b;
    nextVertex[idx] = head[a];
    head[a] = idx++;
}

bool topsort()
{
    int qHeadIdx = 0, qTailIdx = -1; //队头, 队尾
    //从前往后遍历入度为0的点, 插入队列
    for (int i = 1; i <= n; i++)
        if (!d[i])
            q[++qTailIdx] = i; //数组模拟队列, 入队是尾指针+1
    while (qHeadIdx <= qTailIdx) //如果头指针小于尾指针
    {
        int t = q[qHeadIdx++]; //取队列头元素,, 出队只是把头指针往后移动一位
        //遍历t的临边, 空指针初始化为-1
        for (int i = head[t]; i != -1; i = nextVertex[i])
        {
            int j = edge[i]; //取到出边
            d[j]--; //入度减一
            if (d[j] == 0)
                q[++qTailIdx] = j; //如若入度为0, 压入队列
        }
    }
    //判断是否所有顶点都入队
    //也就是头指针qTailIdx是否等于n-1, 即所有点都进入队列了
    return qTailIdx == n-1;
}

int main()
{
    cin >> n >> m; //读入n,m
    memset(head, -1, sizeof(head)); //初始化Head数组指向-1顶点

    For(i, 0, m)
    {
        int a, b;
        cin >> a >> b;
        add(a, b); //插入边
        d[b]++; //更新入度
    }

    if (topsort())
    { //数组队列里面的元素就是拓扑序
        for (int i = 0; i < n; i++) printf("%d ", q[i]);
    }
}

```

Python

Python solution pending

```
    }  
    else  
        puts("-1");  
    return 0;  
}
```

NQ148 Dijkstra求最短路(1) AcWing 849

给定一个n个点m条边的有向图，图中可能存在重边和自环，所有边权均为正值。请你求出1号点到n号点的最短距离，如果无法从1号点走到n号点，则输出-1。

输入 第一行包含整数n和m。接下来m行每行包含三个整数x, y, z，表示存在一条从点x到点y的有向边，边长为z。

输出 输出一个整数，表示1号点到n号点的最短距离。如果路径不存在，则输出-1。

来源 AcWing 849

输入	输出
3 3	3
1 2 2	
2 3 1	
1 3 4	

提示 [原题链接](#)

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 510; // 500*500
const int INF = 0x3f3f3f3f;

int g[N][N];
int dist[N];
bool st[N];

int n, m;

int Dijkstra()
{
    memset(dist, 0x3f, sizeof dist); // 初始化距离 0x3f代表无限大

    dist[1] = 0; // 第一个点到自身的距离为0

    // 有n个点, 迭代n次
    for (int i = 0; i < n; i++)
    {
        int t = -1; // 当前访问点的编号初始为-1

        for (int j = 1; j <= n; j++) // 从点1到点n开始计算dist
            if (!st[j] && (t == -1 || dist[t] > dist[j]))
                t = j; // 更新点t, 直到找到dist最小的点

        st[t] = true; // 标记此点确定

        for (int j = 1; j <= n; j++) // 每个点都要遍历所有点一遍
            dist[j] = min(dist[j], dist[t] + g[t][j]); // 遍历所有的点, 求从点t到点j的最小值
    }

    if (dist[n] == INF) return -1;
    return dist[n]; // 返回第n个点的最短距离
}

int main()
{
    cin >> n >> m;

    memset(g, 0x3f, sizeof g);

    while (m--)
    {
        int x, y, z;
        cin >> x >> y >> z;
        g[x][y] = min(g[x][y], z); // 取重边和自环的最小值, 保留一条边
    }

    cout << Dijkstra() << endl;

    return 0;
}

```

Python

Python solution pending

NQ149 Dijkstra求最短路(2) AcWing 850

给定一个 n 个点 m 条边的有向图，图中可能存在重边和自环，所有边权均为非负值。请你求出1号点到 n 号点的最短距离，如果无法从1号点走到 n 号点，则输出 -1 。

输入

第一行包含整数 n 和 m 。接下来 m 行每行包含三个整数 x, y, z ，表示存在一条从点 x 到点 y 的有向边，边长为 z 。

输出

输出一个整数，表示1号点到 n 号点的最短距离。如果路径不存在，则输出 -1 。

来源

AcWing 850

输入

```
3 3
1 2 2
2 3 1
1 3 4
```

输出

```
3
```

提示 [原题链接](#) [Y总讲解](#) [Y总代码](#)

C++

```

#include <algorithm>
#include <cstring>
#include <iostream>
#include <queue>

using namespace std;

typedef pair<int, int> PII;

const int N = 1e6 + 10;

int n, m; //行, 列
int h[N], w[N], e[N], ne[N], idx; //数组模拟链表
int dist[N]; //距离矩阵
bool st[N]; //状态矩阵

//建边函数 (数组模拟链表)
void add(int a, int b, int c) {
    e[idx] = b, w[idx] = c, ne[idx] = h[a], h[a] = idx++;
}

//改进版, 使用小根堆
int dijkstra() {
    //初始化距离矩阵
    memset(dist, 0x3f, sizeof dist);
    dist[1] = 0;
    //调用优先队列, 以及greater算子, 创建小根堆
    priority_queue<PII, vector<PII>, greater<PII>> heap;

    //按照距离排序, 因此第一个元素是距离, 第二个元素是顶点编号
    heap.push({0, 1});

    // BFS计算最短距离
    while (heap.size()) {
        //取堆头元素
        auto t = heap.top();
        heap.pop();

        // 取顶点编号、顶点距离
        int ver = t.second, distance = t.first;

        // 顶点遍历过就跳过, 不展开改顶点
        if (st[ver]) continue;
        st[ver] = true;

        //遍历所有相邻顶点
        for (int i = h[ver]; i != -1; i = ne[i]) {
            int j = e[i];
            //判断dist[ver]+w[i]是否比原来顶点j的距离大
            if (dist[j] > dist[ver] + w[i]) {
                dist[j] = dist[ver] + w[i]; //更新顶点j的距离值
                heap.push({dist[j], j}); //把更小的距离值入堆
            }
        }
    }

    //如果第n点距离值没被更新, 表明不可达
    if (dist[n] == 0x3f3f3f3f) return -1;
    //返回点n的距离值
    return dist[n];
}

int main() {
    //读入行n, 列m

```

Python

Python solution pending

```
scanf("%d%d", &n, &m);
//把邻接链表的头指针化为-1, 表明当前为空, 不指向任何链表
memset(h, -1, sizeof h);
//处理m条边
while (m--) {
    int a, b, c;
    scanf("%d%d%d", &a, &b, &c);
    add(a, b, c); //调用add建图
}
//调用改进版的dijkstra()
cout << dijkstra() << endl;

return 0;
}
```

NQ150 spfa求最短路 AcWing 851

给定一个 n 个点 m 条边的有向图，图中可能存在重边和自环，边权可能为负数。请你求出 1 号点到 n 号点的最短距离，如果无法从 1 号点走到 n 号点，则输出 impossible。数据保证不存在负权回路。

输入 第一行包含整数 n 和 m 。接下来 m 行每行包含三个整数 x, y, z ，表示存在一条从点 x 到点 y 的有向边，边长为 z 。数据范围 $1 \leq n, m \leq 10^5$ ，图中涉及边长绝对值均不超过 10000。

输出 输出一个整数，表示 1 号点到 n 号点的最短距离。如果路径不存在，则输出 impossible。

来源 AcWing 851

输入	输出
3 3	2
1 2 5	
2 3 -3	
1 3 4	

提示 原题链接 Y总讲解 Y总代码

C++

```

#include <queue>
#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 100007;

int n,m;
int h[N], e[N], w[N], ne[N], idx;
int q[N], dist[N];
bool st[N];

void add(int a, int b, int c) // 添加一条边a->b, 边权为c
{
    e[idx] = b, w[idx] = c, ne[idx] = h[a], h[a] = idx ++;
}

int spfa() // 求1号点到n号点的最短路距离, 如果从1号点无法走到n号点则返回-1
{
    int hh = 0, tt = 0;
    memset(dist, 0x3f, sizeof dist);
    dist[1] = 0;
    q[tt ++ ] = 1;
    st[1] = true;

    while (hh != tt)
    {
        int t = q[hh ++ ];
        if (hh == N) hh = 0;
        st[t] = false;

        for (int i = h[t]; i != -1; i = ne[i])
        {
            int j = e[i];
            if (dist[j] > dist[t] + w[i])
            {
                dist[j] = dist[t] + w[i];
                if (!st[j]) // 如果队列中已存在j, 则不需要
                    将j重复插入
                {
                    q[tt ++ ] = j;
                    if (tt == N) tt = 0;
                    st[j] = true;
                }
            }
        }
    }

    if (dist[n] == 0x3f3f3f3f) return -1;
    return dist[n];
}

int main()
{
    scanf("%d%d", &n, &m);
    memset(h, -1, sizeof h);
    while (m -- )
    {
        int a, b, c;

```

Python

Python solution pending


```

scanf("%d%d%d", &a, &b, &c);
add(a, b, c);
}

int t = spfa();

if (t == -1) puts("impossible");
else printf("%d\n", t);

return 0;
}

```

NQ151 Floyd求最短路 AcWing 854

给定一个 n 个点 m 条边的有向图，图中可能存在重边和自环，边权可能为负数。再给定 k 个询问，每个询问包含两个整数 x 和 y ，表示查询从点 x 到点 y 的最短距离，如果路径不存在，则输出 impossible。数据保证图中不存在负权回路。数据范围 $1 \leq n \leq 200$, $1 \leq k \leq n^2$, $1 \leq m \leq 20000$ ，图中涉及边长绝对值均不超过 10000。

输入 第一行包含三个整数 n, m, k 。接下来 m 行，每行包含三个整数 x, y, z ，表示存在一条从点 x 到点 y 的有向边，边长为 z 。接下来 k 行，每行包含两个整数 x, y ，表示询问点 x 到点 y 的最短距离。

输出 共 k 行，每行输出一个整数，表示询问的结果，若询问两点间不存在路径，则输出 impossible。

来源 AcWing 854

输入

```

3 3 2
1 2 1
2 3 2
1 3 1
2 1
1 3

```

输出

```

impossible
1

```

提示 原题链接 Y总讲解 Y总代码

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 207, INF = 0x3f3f3f3f;

int n, m, k; // k组询问
int d[N][N];

void floyd()
{
    // 用动态规划分析得出这个递推公式, 使用三重循环
    for (int k = 1; k <= n; k++)
        for (int i = 1; i <= n; i++)
            for (int j = 1; j <= n; j++)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
}

int main()
{
    scanf("%d%d%d", &n, &m, &k);

    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            if (i == j) d[i][j] = 0;
            else d[i][j] = INF;

    // 处理每条边, 去掉重边和自环
    while (m--)
    {
        int a, b, c;
        scanf("%d%d%d", &a, &b, &c);
        d[a][b] = min(d[a][b], c);
    }

    floyd();

    // k组查询
    while (k--)
    {
        int a, b;
        scanf("%d%d", &a, &b);
        int t = d[a][b];
        if (t > INF / 2) puts("impossible");
        else printf("%d\n", t);
    }

    return 0;
}

```

Python

Python solution pending

NQ152 Prim算法求最小生成树 AcWing 858

给定一个 n 个点 m 条边的无向图, 图中可能存在重边和自环, 边权可能为负数。求最小生成树的树边权重之和, 如果最小生成树不存在则输出 impossible。给定一张边带权的无向图 $G = (V, E)$, 其中 V 表示图中点的集合, E 表示图中边的集合, $n = |V|$, $m = |E|$ 。由 V 中的全部 n 个顶点和 E 中 $n - 1$ 条边构成的无向连通子图

被称为 G 的一棵生成树，其中边的权值之和最小的生成树被称为无向图 G 的最小生成树。数据范围 $1 \leq n \leq 500, 1 \leq m \leq 10^5$ ，图中涉及边的边权的绝对值均不超过 10000。

输入

第一行包含两个整数 n 和 m 。接下来 m 行，每行包含三个整数 u, v, w ，表示点 u 和点 v 之间存在一条权值为 w 的边。

输出

共一行，若存在最小生成树，则输出一个整数，表示最小生成树的树边权重之和，如果最小生成树不存在则输出 impossible。

来源

AcWing 858

输入

```
4 5
1 2 1
1 3 2
1 4 3
2 3 2
3 4 4
```

输出

```
6
```

提示 [原题链接](#) [Y总讲解](#) [参考题解](#) [Y总代码](#)

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 507, INF = 0x3f3f3f3f;

int n, m;
int g[N][N];
int dist[N];
bool st[N];

int prim()
{
    memset(dist, 0x3f, sizeof dist);

    int res = 0;
    for (int i = 0; i < n; i++)
    {
        //第一个点
        int t = -1;
        for (int j = 1; j <= n; j++)
            if (!st[j] && (t == -1 || dist[t] > dist[j]))
                t = j;

        if (i && dist[t] == INF) return INF; //没有最小生成树, 不连通

        if (i) res += dist[t];
        st[t] = true;

        for (int j = 1; j <= n; j++) dist[j] = min(dist[j], g[t][j]);
    }
    return res;
}

int main()
{
    scanf("%d%d", &n, &m);

    memset(g, 0x3f, sizeof g);

    while (m--)
    {
        int a, b, c;
        scanf("%d%d%d", &a, &b, &c);
        g[a][b] = g[b][a] = min(g[a][b], c); //无向图, 去重
    }

    int t = prim();
    if (t == INF) puts("impossible");
    else printf("%d\n", t);

    return 0;
}

```

Python

Python solution pending

给定一个 n 个点 m 条边的无向图，图中可能存在重边和自环，边权可能为负数。求最小生成树的树边权重之和，如果最小生成树不存在则输出impossible。给定一张边带权的无向图 $G=(V, E)$ ，其中 V 表示图中点的集合， E 表示图中边的集合， $n = |V|$ ， $m = |E|$ 。由 V 中的全部 n 个顶点和 E 中 $n - 1$ 条边构成的无向连通子图被称为 G 的一棵生成树，其中边的权值之和最小的生成树被称为无向图 G 的最小生成树。数据范围 $1 \leq n \leq 10^5, 1 \leq m \leq 2 \times 10^5$ 图中涉及边的边权的绝对值均不超过1000

输入

第一行包含两个整数 n 和 m 。接下来 m 行，每行包含三个整数 u 、 v 、 w ，表示点 u 和点 v 之间存在一条权值为 w 的边。

输出

共一行，若存在最小生成树，则输出一个整数，表示最小生成树的树边权重之和，如果最小生成树不存在则输出impossible。

来源

AcWing 859

输入

```
4 5
1 2 1
1 3 2
1 4 3
2 3 2
3 4 4
```

输出

```
6
```

提示 [原题链接](#) [Y总讲解](#) [参考题解](#) [Y总代码](#)

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 100007, M = 2000007, INF = 0x3f3f3f3f;

int n,m;
int p[N]; //并查集

struct Edge
{
    int a, b, w;
    bool operator< (const Edge &a) const
    {
        return w < a.w;
    }
} edges[M]; //重载<, 安装权重w比大小

int find(int x) // 并查集
{
    if (p[x] != x) p[x] = find(p[x]);
    return p[x];
}

int kruskal()
{
    //1.排序
    sort(edges, edges+m);
    //并查集初始化
    for (int i = 1; i <= n; i++) p[i] = i;

    int res = 0, cnt = 0;
    for (int i = 0; i < m; i++)
    {
        int a = edges[i].a, b = edges[i].b, w = edges[i].w;
        a = find(a), b = find(b);
        if (a != b)
        {
            p[a] = b;
            res += w;
            cnt++;
        }
    }
    if (cnt < n-1) return INF;
    return res;
}

int main()
{
    scanf("%d%d", &n, &m);

    for (int i = 0; i < m; i++)
    {
        int a, b, w;
        scanf("%d%d%d", &a, &b, &w);
        edges[i] = {a, b, w};
    }

    int t = kruskal();

    if (t == INF) puts("impossible");
    else printf("%d\n", t);
}

```

Python

Python solution pending

```
    return 0;  
}
```

NQ154 染色法判定二分图 AcWing 860

给定一个 n 个点 m 条边的无向图，图中可能存在重边和自环。请你判断这个图是否是二分图。

输入 第一行包含两个整数 n 和 m 。接下来 m 行，每行包含两个整数 u 和 v ，表示点 u 和点 v 之间存在一条边。数据范围 $1 \leq n, m \leq 10^5$

输出 如果给定图是二分图，则输出Yes，否则输出No。

来源 AcWing 860

输入	输出
4 4	Yes
1 3	
1 4	
2 3	
2 4	

提示 [原题链接](#) [视频讲解](#) [参考题解](#) [Y总代码](#)

C++

```

#include <iostream>
#include <algorithm>
#include <cstring>
using namespace std;
//无向图
const int N=100007,M=200007;//两倍的边
int n,m;
int h[N],e[M],ne[M],idx;
int color[N];//0表示未访问, 1表示颜色1, 2表示颜色2.

void add(int a, int b){
    e[idx]=b, ne[idx]=h[a], h[a]=idx++; //数组模拟链表add操作
}

bool dfs(int u, int c){
    color[u]=c;

    for (int i=h[u]; i!=-1; i=ne[i])
    {
        int j=e[i]; //取i对应的另外一个顶点编号
        if (!color[j]) {
            if (!dfs(j,3-c)) return false;//颜色是1或者2切换, 所以
            //用3-color可以交换两种颜色
        }
        else if (color[j]==c) return false;//端点颜色相同
    }
    return true;
}

int main(){

    scanf("%d%d",&n,&m);
    memset(h,-1,sizeof h);//初始化头结点
    //建图
    while (m--)
    {
        int a,b;
        scanf("%d%d",&a,&b);
        add(a,b); add(b,a);
    }
    bool flag = true;//表示染色有矛盾
    for (int i=1; i<=n; i++)
        if (!color[i]) {
            if (!dfs(i,1))
            {
                flag = false;
                break;
            }
        }
    if (flag) puts("Yes"); else puts ("No");

    return 0;
}

```

Python

Python solution pending